TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
DS201 – Deep Learning trong Khoa học Dữ liệu

BÁO CÁO LAB 2: MẠNG NEURAL TÍCH CHẬP

Họ và tên: Bùi Thị Thanh Tuyền MSSV: 24521956

1 Thiết lập chung

Bộ dữ liệu là VinaFood21 với 21 lớp món ăn Việt Nam. Thư mục `train` có 10,044 ảnh, thư mục `test` có 6,682 ảnh. Vì bản tải về không có thư mục `val`, tôi tách 10% ảnh của `train` bằng `torch.randperm` với seed 42, được 9,040 ảnh huấn luyện và 1,004 ảnh kiểm định. Cả bốn bài dùng chung một phép chia này. Số ảnh mỗi lớp trong tập test dao động từ 143 đến 607.

Ảnh được chuẩn hoá bằng mean và std của ImageNet. Tập huấn luyện dùng `RandomResizedCrop(scale=` `1.0))`, lật ngang, xoay tối đa 15 độ và `ColorJitter(0.2, 0.2, 0.2)`. Tập kiểm định và tập test chỉ resize rồi chuẩn hoá. Kích thước ảnh là 64×64 cho Bài 1 và Bài 2, 96×96 cho Bài 3, 224×224 cho Bài 4.

Tất cả mô hình dùng Adam, hàm mất mát `CrossEntropyLoss`, batch size 32 và seed 42. Bài 1 đến Bài 3 học với learning rate 10^{-3} trong 100 epoch. Bài 4 chạy 15 epoch, learning rate 10^{-3} cho feature extraction và 10^{-4} cho fine-tuning. Sau mỗi epoch tôi tính macro-F1 trên tập kiểm định và giữ lại trọng số có macro-F1 kiểm định cao nhất; các số liệu trên tập test đều lấy từ trọng số này. Precision, recall và F1 là trung bình macro trên 21 lớp.

Mô hình được huấn luyện trên một lát MIG 2g.35gb của GPU H200. Thời gian mỗi epoch gồm cả lượt đánh giá trên tập kiểm định; với các mạng nhỏ, thời gian này bị chi phối chủ yếu bởi việc đọc và biến đổi ảnh.

2 Bài 1: LeNet-5

Mạng giữ nguyên 7 layer C1, S2, C3, S4, F5, F6, OUT với ảnh vào 64×64×3, 21 lớp, hàm kích hoạt ReLU. Hai bản được so sánh: S2 và S4 là `AvgPool2d(2, 2)`, và bản thay bằng `MaxPool2d(2, 2)`. Tổng số tham số là 339,421 cho cả hai bản (lớp pooling không có tham số). Bảng 1 là shape và số tham số từng layer.

Layer	Output	Tham số
C1	6×60×60	456
S2	6×30×30	0
C3	16×26×26	2,416
S4	16×13×13	0
Flatten	2704	0
F5	120	324,600
F6	84	10,164
OUT	21	1,785
Tổng		339,421

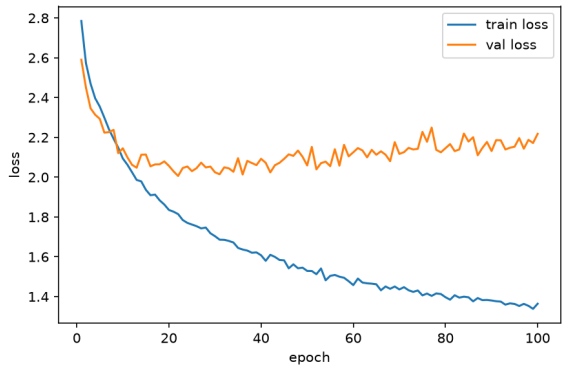
Table 1: Shape và tham số của LeNet-5 với ảnh 64×64×3.

Mô hình	Accuracy	Precision	Recall	F1-macro
LeNet-5, AvgPool	0.3257	0.3205	0.3169	0.3133
LeNet-5, MaxPool	0.3358	0.3376	0.3369	0.3296

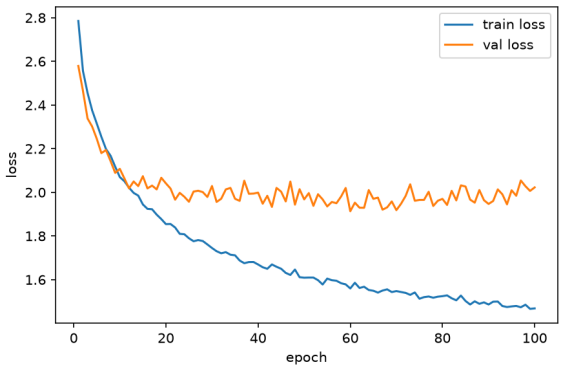
Table 2: Kết quả của Bài 1 trên tập test.

Bản MaxPool cao hơn bản AvgPool 1.6 điểm macro-F1 (0.3296 so với 0.3133). Mỗi cấu hình chỉ chạy một lần với một seed nên chênh lệch này nhỏ và chưa đủ để kết luận MaxPool tốt hơn. Cả hai bản đều thấp: ở epoch cuối, độ chính xác huấn luyện chỉ khoảng 0.53 đến 0.56, tức mô hình chưa khớp nổi tập huấn luyện. Với chỉ 339 nghìn tham số và hai lớp tích chập 6 và 16 filter, LeNet không đủ sức biểu diễn ảnh món ăn có nhiều chi tiết. Loss kiểm định chạm đáy ở epoch 22 (AvgPool, 2.006) và epoch 60 (MaxPool, 1.914) rồi dao động, trong khi các độ đo trên tập kiểm định vẫn tăng chậm đến cuối nên epoch tốt nhất được chọn là 96 và 87.

Các cặp bị nhầm nhiều nhất của LeNet đều là các món có màu trắng hoặc nâu tương tự: bánh ướt bị đoán thành bánh hỏi (104 ảnh với AvgPool, 116 ảnh với MaxPool), há cảo bị đoán thành tàu hũ (68 và 82 ảnh), và bò kho bị lẫn với phá lấu.

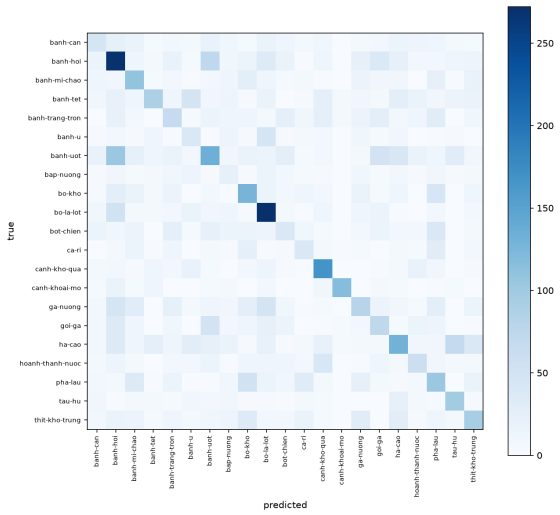


(a) AvgPool

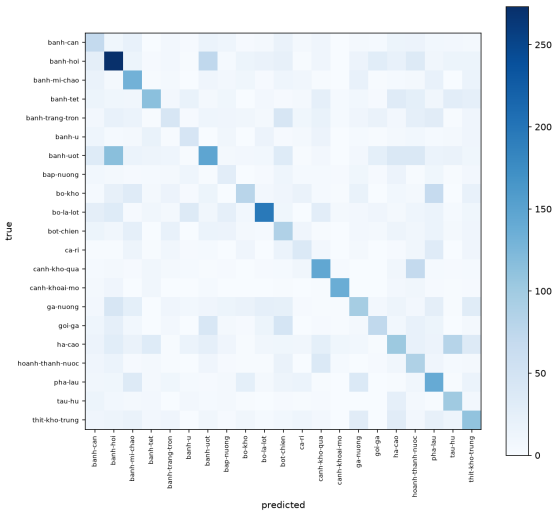


(b) MaxPool

Figure 1: Đường loss huấn luyện và kiểm định của LeNet-5.



(a) AvgPool



(b) MaxPool

Figure 2: Ma trận nhầm lẫn 21×21 của LeNet-5 trên tập test.

3 Bài 2: GoogLeNet rút gọn

Mạng gồm phần stem, bốn Inception block, `AdaptiveAvgPool2d(1)` thay cho `Flatten`, dropout 0.4 và một lớp `Linear` 21 lớp. Stem gồm conv 3×3 (3→32), max pool 2×2 và conv 3×3 (32→64). Mỗi Inception block gồm bốn nhánh 1×1, 1×1 rồi 3×3, 1×1 rồi 5×5, max pool 3×3 rồi 1×1, nối bằng `torch.cat(..., dim=1)`. Các conv trong mạng có BatchNorm; max pool 2×2 đặt sau block 2 và block 3. Không dùng auxiliary classifier.

Hai block tham chiếu trong đề (không BatchNorm) cho đúng số liệu: block $(1, 16, 16, 24, 4, 8, 8)$ ra 56 channel với 4,376 tham số, và block $(56, 32, 32, 48, 8, 16, 16)$ ra 112 channel với 22,104 tham số. Bảng 3 là cấu hình các block trong mô hình huấn luyện; số channel ra là tổng $f_1 + f_3 + f_5 + f_p$, đã tính tay và khớp với giá trị in từ code.

Block	Channel vào	$(f_1, f_{3r}, f_3, f_{5r}, f_5, f_p)$	Channel ra	Output	Tham số
1	64	$(32, 32, 48, 8, 16, 16)$	112	112×32×32	22,960
2	112	$(64, 48, 96, 12, 32, 32)$	224	224×32×32	69,112
3	224	$(96, 64, 128, 16, 48, 48)$	320	320×16×16	143,904
4	320	$(128, 96, 160, 24, 64, 64)$	416	416×8×8	277,552

Table 3: Các Inception block của GoogLeNet rút gọn (output là shape ngay sau block, trước max pool).

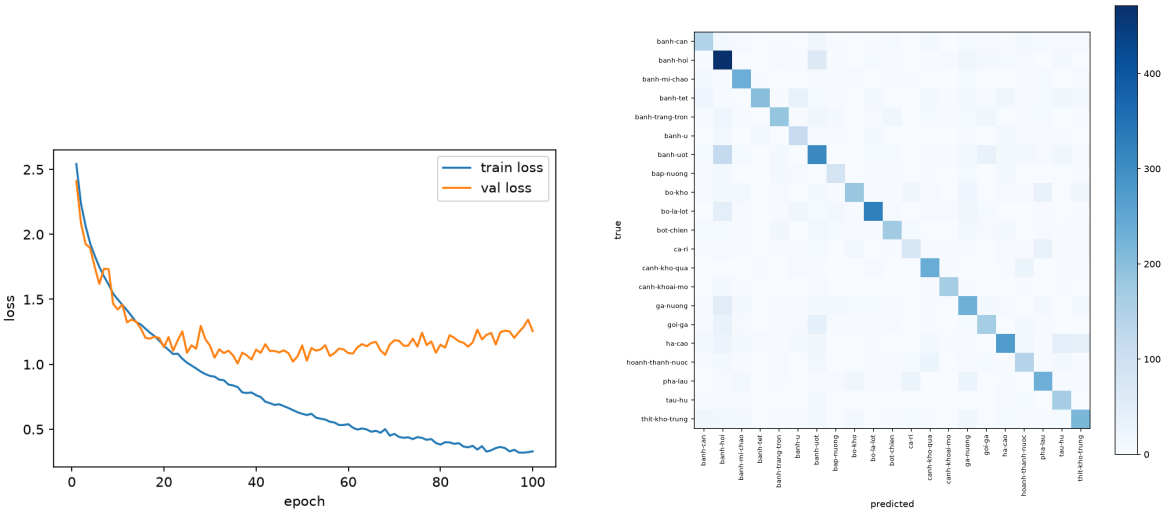
Stem có 19,488 tham số, bốn block có 513,528 tham số, lớp phân loại có $416 \times 21 + 21 =$ 8,757 tham số. Tổng cộng 541,773 tham số.

Mô hình	Accuracy	Precision	Recall	F1-macro
GoogLeNet rút gọn	0.6494	0.6610	0.6491	0.6482

Table 4: Kết quả của Bài 2 trên tập test.

So với LeNet-5, GoogLeNet có số tham số gấp khoảng 1.6 lần (541,773 so với 339,421), thời gian mỗi epoch dài hơn (2.90 giây so với 1.17 giây) và macro-F1 cao hơn rất nhiều: 0.6482 so với 0.3133 (AvgPool) và 0.3296 (MaxPool). Độ lệch lớn này đến từ việc mạng sâu hơn, nhiều channel hơn, dùng BatchNorm, và đặc biệt là không có lớp fully connected 324,600 tham số: sau khi gộp bằng global average pooling, lớp phân loại chỉ còn 8,757 tham số. Tôi không tách riêng từng yếu tố nên không thể nói yếu tố nào đóng góp nhiều nhất. Loss kiểm định thấp nhất ở epoch 36 (1.006), sau đó tăng nhẹ trong khi loss huấn luyện tiếp tục giảm xuống 0.33; macro-F1 kiểm định vẫn cải thiện chậm nên epoch tốt nhất được chọn là 81.

Độ chính xác trên tập kiểm định (0.708 ở epoch cuối) cao hơn trên tập test (0.649). Tập kiểm định được lấy từ `train` nên có phân phối ảnh gần với dữ liệu huấn luyện hơn tập test; số liệu trên test là con số đáng tin hơn.



(a) Đường loss (b) Ma trận nhầm lẫn

Figure 3: GoogLeNet rút gọn.

4 Bài 3: ResNet-18 huấn luyện từ đầu

ResNet-18 gồm stem (conv 7×7 stride 2, BatchNorm, ReLU, max pool), bốn stage mỗi stage hai BasicBlock, global average pooling và `Linear(512, 21)`. Shortcut là identity khi shape khớp và là conv 1×1 cùng BatchNorm khi đổi channel hoặc dùng stride 2. Mạng có 11,187,285 tham số, bằng với `torchvision.models.resnet18` khi đổi lớp cuối sang 21 lớp. Với ảnh vào 96×96, output của layer4 là $512 \times 3 \times 3$. Mô hình không dùng trọng số pretrained và được chạy hai lần: có augmentation và không có augmentation (tập huấn luyện chỉ resize và chuẩn hoá).

Stage	Output	Tham số
stem	64×24×24	9,536
layer1	64×24×24	147,968
layer2	128×12×12	525,568
layer3	256×6×6	2,099,712
layer4	512×3×3	8,393,728
fc	21	10,773
Tổng		11,187,285

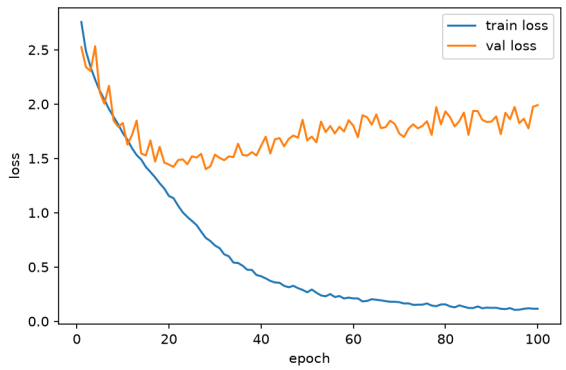
Table 5: Shape và tham số của ResNet-18 với ảnh 96×96.

Mô hình	Accuracy	Precision	Recall	F1-macro
ResNet-18, có augmentation	0.5545	0.5655	0.5436	0.5448
ResNet-18, không augmentation	0.4500	0.4408	0.4370	0.4370

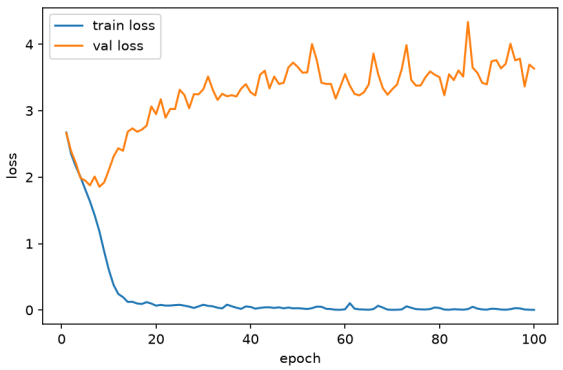
Table 6: Kết quả của Bài 3 trên tập test.

Augmentation làm macro-F1 tăng từ 0.4370 lên 0.5448, tức 10.8 điểm. Đường loss cho thấy rõ nguyên nhân (hình 4). Khi không có augmentation, loss huấn luyện giảm rất nhanh, xuống 0.61 ở epoch 10 và 0.003 ở epoch 100 với độ chính xác huấn luyện 100%, trong khi loss kiểm định đạt đáy 1.855 ở epoch 8 rồi tăng liên tục lên 3.63. Hai đường tách nhau từ khoảng epoch 5 đến 6 và đó cũng là lúc mô hình bắt đầu học thuộc dữ liệu. Khi có augmentation, hai đường đi sát nhau đến khoảng epoch 8 (1.88 và 1.85), bắt đầu tách ra từ epoch 10 (1.73 và 1.83), loss kiểm định đạt đáy 1.401 ở epoch 28 rồi tăng nhẹ lên khoảng 1.8 đến 2.0, còn loss huấn luyện giảm xuống 0.12. Như vậy augmentation làm chậm quá trình học thuộc và dời điểm overfitting từ epoch 8 sang epoch 28, nhưng không loại bỏ được nó.

ResNet-18 có 11.2 triệu tham số, gấp 33 lần LeNet-5 và 21 lần GoogLeNet rút gọn, nhưng chỉ có chưa đến 9,100 ảnh huấn luyện. Kết quả 0.5448 thấp hơn GoogLeNet (0.6482) không có nghĩa là cài sai, vì số tham số khớp với torchvision; nguyên nhân phù hợp với đường loss ở trên là mô hình lớn huấn luyện từ đầu trên tập nhỏ dễ overfit.

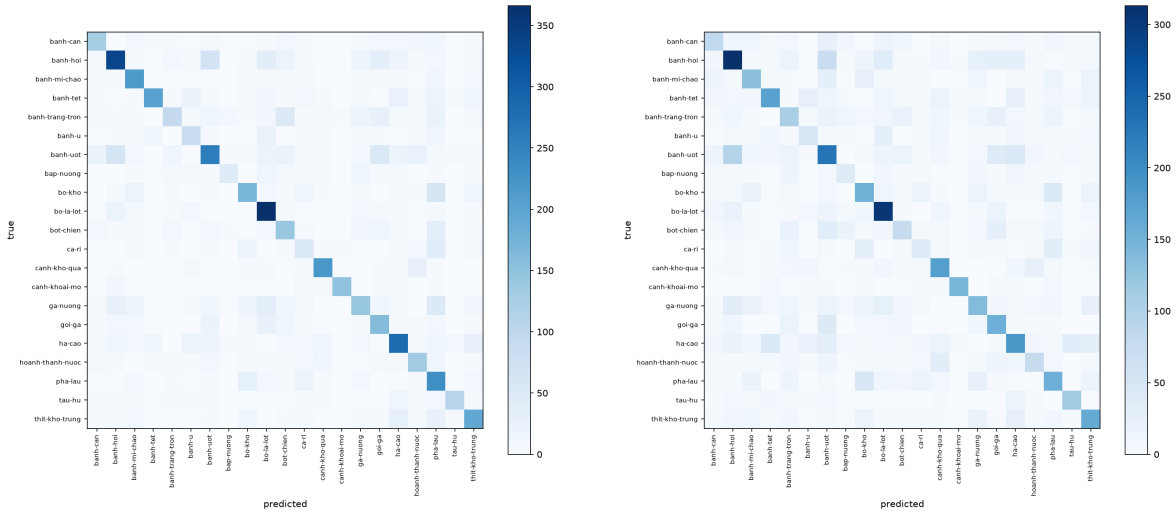


(a) Có augmentation



(b) Không augmentation

Figure 4: Đường loss huấn luyện và kiểm định của ResNet-18.



(a) Có augmentation

(b) Không augmentation

Figure 5: Ma trận nhầm lẫn của ResNet-18 trên tập test.

5 Bài 4: Fine-tune ResNet-50 pretrained

Mô hình dùng `microsoft/resnet-50` từ HuggingFace, lấy phần backbone và gắn `nn.Linear(2048, 21)`. Backbone có 23,508,032 tham số, toàn mô hình có 23,551,061 tham số. Ảnh được resize về 224×224 và chuẩn hoá bằng mean, std của ImageNet. Hai chế độ được chạy, và optimizer chỉ nhận các tham số có `requires_grad = True`:

- Feature extraction: đóng băng backbone, chỉ học classifier (43,029 tham số), Adam, learning rate 10^{-3}.
- Fine-tuning: cập nhật toàn bộ 23,551,061 tham số, Adam, learning rate 10^{-4}.

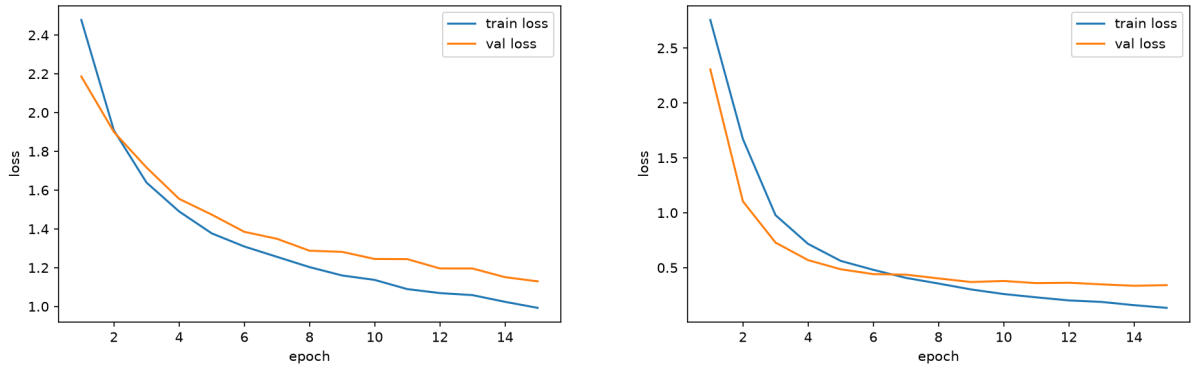
Chế độ	Tham số học	Accuracy	Precision	Recall	F1-macro
Feature extraction	43,029	0.6182	0.6455	0.6228	0.6229
Fine-tuning	23,551,061	0.8548	0.8608	0.8560	0.8567

Table 7: Kết quả của Bài 4 trên tập test.

Fine-tuning đạt macro-F1 0.8567, cao hơn feature extraction 23.4 điểm và cao hơn ResNet-18 huấn luyện từ đầu (0.5448) 31.2 điểm. ResNet-50 đã học các đặc trưng thị giác tổng quát (cạnh, kết cấu, bộ phận vật thể) từ ImageNet nên chỉ cần điều chỉnh nhẹ với learning rate nhỏ, trong khi ResNet-18 phải học toàn bộ từ khoảng 9,000 ảnh. Feature extraction (0.6229) vẫn cao hơn ResNet-18 từ đầu dù chỉ học một lớp tuyến tính, vì đặc trưng 2048 chiều đã tốt; nhưng nó thấp hơn fine-tuning rất nhiều vì ImageNet không có phân bố giống ảnh món ăn Việt Nam, các channel của backbone cần được tinh chỉnh để phân biệt

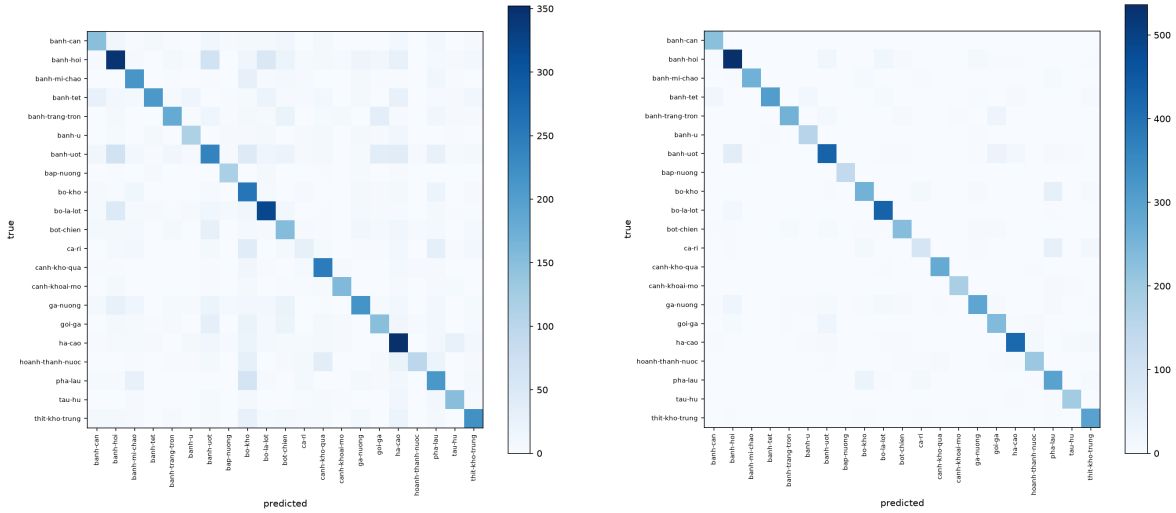
các món có hình dạng và màu sắc gần nhau. Ở chế độ feature extraction, loss kiểm định vẫn giảm đến epoch 15 (1.129) nên chạy thêm epoch có thể còn cải thiện; ở fine-tuning, loss kiểm định đạt đáy 0.335 ở epoch 14 nên 15 epoch là đủ.

Cái giá của fine-tuning là thời gian: 17.65 giây mỗi epoch, so với 6.87 giây của feature extraction và 2.48 giây của ResNet-18 từ đầu.



(a) Feature extraction (b) Fine-tuning

Figure 6: Đường loss huấn luyện và kiểm định của ResNet-50.



(a) Feature extraction (b) Fine-tuning

Figure 7: Ma trận nhầm lẫn của ResNet-50 trên tập test.

6 Bảng tổng hợp

Mô hình	Resize	Tham số	Giây/epoch	Accuracy	macro-F1
LeNet-5 (AvgPool)	64×64	339,421	1.17	0.3257	0.3133
LeNet-5 (MaxPool)	64×64	339,421	1.16	0.3358	0.3296
GoogLeNet rút gọn	64×64	541,773	2.90	0.6494	0.6482
ResNet-18 từ đầu, có augmentation	96×96	11,187,285	2.48	0.5545	0.5448
ResNet-18 từ đầu, không augmentation	96×96	11,187,285	2.28	0.4500	0.4370
ResNet-50, feature extraction	224×224	23,551,061	6.87	0.6182	0.6229
ResNet-50, fine-tuning	224×224	23,551,061	17.65	0.8548	0.8567

Table 8: Tổng hợp kết quả trên tập test. Số tham số của hai dòng ResNet-50 là tổng; số tham số được cập nhật lần lượt là 43,029 và 23,551,061.

Lớp	Số ảnh	LeNet Avg	LeNet Max	GoogLeNet	ResNet-18	R50 FE	R50 FT
banh-can	241	0.204	0.248	0.596	0.564	0.629	0.897
banh-hoi	607	0.402	0.427	0.639	0.588	0.588	0.848
banh-mi-chao	289	0.336	0.381	0.763	0.657	0.690	0.927
banh-tet	364	0.313	0.373	0.691	0.655	0.688	0.903
banh-trang-tron	309	0.223	0.177	0.656	0.379	0.669	0.870
banh-u	176	0.202	0.259	0.602	0.463	0.683	0.880
banh-uot	560	0.271	0.300	0.549	0.500	0.464	0.796
bap-nuong	143	0.131	0.175	0.577	0.391	0.795	0.972
bo-kho	338	0.343	0.289	0.622	0.519	0.536	0.775
bo-la-lot	458	0.509	0.494	0.760	0.689	0.684	0.901
bot-chien	276	0.161	0.258	0.669	0.468	0.516	0.843
ca-ri	171	0.225	0.228	0.474	0.297	0.233	0.579
canh-kho-qua	290	0.502	0.443	0.767	0.742	0.749	0.950
canh-khoai-mo	198	0.688	0.719	0.845	0.809	0.819	0.931
ga-nuong	372	0.245	0.284	0.590	0.427	0.640	0.816
goi-ga	290	0.250	0.300	0.598	0.488	0.534	0.787
ha-cao	480	0.304	0.241	0.642	0.596	0.651	0.892
hoanh-thanh-nuoc	240	0.266	0.265	0.586	0.520	0.546	0.859
pha-lau	352	0.272	0.338	0.656	0.475	0.548	0.760
tau-hu	203	0.426	0.405	0.668	0.639	0.728	0.929
thit-kho-trung	325	0.305	0.318	0.662	0.573	0.693	0.876

Table 9: F1 từng lớp trên tập test. ResNet-18 là bản có augmentation; R50 FE và R50 FT là feature extraction và fine-tuning của ResNet-50.

7 Trả lời câu hỏi kiểm tra nhanh

B.1. Từ ảnh đến feature map

Câu 1. Filter 3×3 trên input 64 channel có $3 \times 3 \times 64 = 576$ trọng số (577 tham số nếu tính bias). Một filter cho ra đúng 1 channel; muốn có n channel ra thì dùng n filter.

Câu 2. Số tham số của `cnv` không đổi, vẫn là 448, vì filter được dùng chung trên toàn ảnh. Với `mlp`, số tham số tăng theo tích số input và số output: ở ảnh 512×512, input là $512 \cdot 512 \cdot 3 = 786{,}432$ và output là $512 \cdot 512 \cdot 16 = 4{,}194{,}304$, nên cần $786{,}432 \times 4{,}194{,}304 = 3{,}298{,}534{,}883{,}328$ tham số, gấp 4,096 lần con số 805,306,368 của ảnh 64×64.

B.2. Output size, pooling và shape

Câu 1. Output có shape $[32, 16, 64, 64]$ vì $P = (K-1)/2 = 2$ giữ nguyên kích thước. Số tham số là $5 \cdot 5 \cdot 3 \cdot 16 + 16 = 1{,}216$.

Câu 2. `MaxPool2d(2, 2)` có 0 tham số. Ba layer conv 3×3 liên tiếp cho receptive field 7×7.

B.3. LeNet-5

Câu 1. LeNet ở cấu hình 64×64×3, 21 lớp có 339,421 tham số. Riêng F5 có 324,600 tham số, chiếm khoảng 95.6%.

Câu 2. Khi ảnh tăng từ 32×32 lên 64×64, kích thước sau S4 đổi từ $5 \times 5 \times 16 = 400$ lên $13 \times 13 \times 16 = 2{,}704$. Hai conv layer chỉ phụ thuộc số channel và kích thước kernel nên vẫn có 2,872 tham số, còn F5 nối 2,704 giá trị với 120 neuron nên tăng từ 48,120 lên 324,600 tham số. Toàn bộ phần tăng thêm nằm ở F5, kéo tổng từ 62,941 lên 339,421 (gấp 5.39 lần).

Câu 3. Nguyên nhân là layer Dropout trong ví dụ (`nn.Dropout(p=0.5)`): ở chế độ `train()` nó tắt ngẫu nhiên các neuron nên hai lần forward cho kết quả khác nhau, còn ở `eval()` thì Dropout không làm gì. BatchNorm cũng hoạt động khác nhau giữa hai chế độ nhưng không có trong ví dụ này.

B.4. Inception, bottleneck 1×1 và global average pooling

Câu 1. Block $(56, 32, 32, 48, 8, 16, 16)$ cho ra $32 + 48 + 16 + 16 = 112$ channel.

Câu 2. Conv 1×1 không nhìn lân cận nhưng trộn thông tin giữa các channel, nên dùng nó để giảm số channel trước conv 5×5. Chi phí của conv 5×5 tỉ lệ với số channel vào; giảm từ 192 xuống 16 channel thì conv 5×5 chỉ còn nhận 16 channel. Kết quả là trọng

số giảm từ 153,600 xuống 15,872 và số phép nhân giảm từ 120,422,400 xuống 12,443,648, đều khoảng 9.68 lần.

B.5. Residual block và ResNet-18

Câu 1. Block đầu tiên của layer2, layer3 và layer4 cần projection shortcut. Ở các block này nhánh chính đổi số channel (64→128, 128→256, 256→512) và dùng stride 2, nên $F(x)$ khác shape với x và không cộng trực tiếp được; conv 1×1 stride 2 cùng BatchNorm đưa x về đúng shape. Các block còn lại dùng identity.

Câu 2. Dễ xảy ra overfitting. Trong thí nghiệm của tôi, bản không augmentation đạt độ chính xác huấn luyện 100% trong khi loss kiểm định tăng từ 1.855 (epoch 8) lên 3.633 (epoch 100); bản có augmentation cũng bị overfit nhưng muộn hơn (đáy loss kiểm định ở epoch 28) và đạt macro-F1 cao hơn 10.8 điểm.

B.6. Transfer learning với ResNet-50

Câu 1. Cuối backbone có global average pooling đưa mỗi channel về một giá trị, nên `pooler_output` có shape $[B, 2048, 1, 1]$. Số 2048 là số channel của stage cuối: 512×4, trong đó 512 là số channel của conv 3×3 trong Bottleneck và 4 là hệ số nở của Bottleneck.

Câu 2. Ở chế độ feature extraction, optimizer nhận 43,029 tham số ($2048 \times 21 + 21$). Nếu quên lọc `requires_grad` và đưa toàn bộ 23,551,061 tham số vào optimizer, các tham số đã đóng băng không có gradient nên Adam bỏ qua chúng và kết quả huấn luyện gần như không đổi, nhưng optimizer chứa danh sách tham số thừa, và nếu sau đó lỡ mở băng một phần backbone thì phần đó sẽ được học với learning rate 10^{-3} không mong muốn. Tôi lọc tham số khi tạo optimizer.

B.7. Khung huấn luyện và đánh giá

Câu 1. Các lớp trong VinaFood21 không cân bằng: ở tập test, lớp ít nhất có 143 ảnh và lớp nhiều nhất có 607 ảnh. Accuracy là tỉ lệ đoán đúng trên toàn bộ ảnh nên bị chi phối bởi các lớp đông; mô hình đoán kém các lớp ít ảnh vẫn có accuracy cao. Macro-F1 lấy trung bình F1 của từng lớp với trọng số như nhau, và F1 kết hợp cả precision lẫn recall, nên phản ánh chất lượng trên mọi lớp.

Câu 2. Bộ dữ liệu này không có phở, bún bò hay hủ tiếu; các món nước chỉ gồm canh khoai mỡ, canh khổ qua và hoành thánh nước. Các món này có F1 cao ở hầu hết mô hình (canh khoai mỡ 0.84 ở GoogLeNet, canh khổ qua 0.95 ở ResNet-50 fine-tuning), nên giả thuyết rằng món nước hay bị nhầm với nhau không đúng với dữ liệu này. Ở LeNet MaxPool có 70 ảnh canh khổ qua bị đoán thành hoành thánh nước. Các cặp bị nhầm nhiều nhất ở hầu hết mô hình là bánh ướt và bánh hỏi (hai món đều làm từ bột gạo, màu trắng, thường ăn kèm rau và nước chấm; riêng GoogLeNet nhầm 120 ảnh bánh ướt

thành bánh hỏi), bò kho và phá lấu (cùng là món kho màu nâu), và há cảo với tàu hũ. ResNet-50 fine-tuning giảm các nhầm lẫn này nhưng cặp bánh ướt và bánh hỏi vẫn còn 55 ảnh, cặp bò kho và phá lấu còn 46 ảnh.